ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

FINAL INTERNSHIP REPORT

DESIGN AND IMPLEMENTATION OF AN ENTERPRISE KNOWLEDGE MANAGEMENT SYSTEM (INSA-KMS)

HOSTING ORGANIZATION:

Information Network Security Administration (INSA)
Addis Ababa, Ethiopia

Prepared By:

- **Student Name:** Samuel Tenkir
- **Student ID:** UGR/35343/16
- **Department:** Software Engineering

Supervised By:

- **Hosting Organization Immediate Supervisor:** Gudissa , INSA
- **University Department Advisor:** Megersa Daraje , Dept. of CSE, ASTU

Internship Period:

- **Duration:** 45 Working Days
- **Internship Dates:** July 07, 2026 – September 04, 2026
- **Submission Date:** September 19, 2026

EXECUTIVE SUMMARY

Knowledge assets within national security and critical information infrastructure institutions represent vital strategic capital. In large-scale, security-sensitive organizations such as the **Information Network Security Administration (INSA)**, technical intelligence, research papers, vulnerability reports, architectural baselines, standard operating procedures (SOPs), and compliance policies are continuously generated across disparate directorates. However, fragmented storage across disconnected file servers, local drives, email attachments, and ad-hoc chat channels introduces severe operational vulnerabilities, including institutional knowledge leakage during staff attrition, version dissonance, redundant analytical effort, and non-compliance with national data classification mandates.

To address these challenges, this two-month internship project focused on the end-to-end engineering, hardening, and deployment of the **INSA Knowledge Management System (INSA-KMS)**. INSA-KMS is a resilient, enterprise-grade digital repository, semantic search platform, and secure real-time collaboration ecosystem designed specifically for internal organizational use.

The technical architecture is built upon a modular 3-tier topology:

1. **Frontend Presentation Tier:** Engineered using **Next.js 14** (React 18, TypeScript, Tailwind CSS, Radix UI), featuring responsive document libraries, dynamic file explorer trees, built-in PDF.js canvas previewing, debounced multi-user collaboration selectors, and real-time status telemetry.
2. **Backend Services Tier:** Built on **Spring Boot 3.3** (Java 21, Maven), exposing a robust suite of RESTful microservices. The backend implements Spring Security as an **OAuth2 Resource Server** validating cryptographically signed RS256 JSON Web Tokens (JWT) from an enterprise **Keycloak 26.7.2** identity broker, enforcing strict Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) based on organizational clearance levels (PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED). Content ingestion features automated cryptographic integrity hashing (SHA-256) and an asynchronous **Apache Tika** parsing pipeline for full-text extraction.
3. **Data and Streaming Persistence Tier:** Powered by **PostgreSQL 15** with customized GIN (Generalized Inverted Index) indexing on tsvector structures for sub-second full-text search across millions of words. Real-time video conferencing is facilitated via a containerized **Jitsi Meet / Prosody XMPP** cluster secured by short-lived HMAC-SHA256 JWT tokens with fail-closed room isolation.
4. **Audit and Governance Subsystem:** Implemented via Spring Aspect-Oriented Programming (AOP), intercepting every state mutation to log immutable, tamper-evident audit events (Actor UUID, Timestamp, IP Address, Action, Resource, Status).

During the 45 working days of the internship, rigorous testing methodologies were applied, including 20 automated JUnit 5/Mockito integration test suites (100% pass rate), end-to-end PowerShell integration verifications across running Keycloak, Spring Boot, and PostgreSQL containers, and direct RFC 7519 XMPP BOSH protocol validations against the Prosody server.

This report documents the organizational context of INSA, orientation experiences, software engineering methodologies, detailed technical designs, implementation workflows, academic knowledge mapping, personal reflections, self-evaluation, and strategic recommendations for ASTU and INSA.

ACKNOWLEDGEMENT

First and foremost, I express my deepest gratitude to the Almighty God for providing me with health, wisdom, perseverance, and guidance throughout my academic studies and the execution of this challenging internship project.

I extend my sincere appreciation to **Adama Science and Technology University (ASTU)**, specifically the **School of Electrical Engineering and Computing (SoEEC)** and the **Department of Computer Science and Engineering**, for equipping me with the strong theoretical foundation, algorithmic discipline, and engineering problem-solving mindset necessary to contribute effectively to national-scale software projects. Special thanks go to my university department advisor for continuous academic guidance, valuable feedback, and constructive critiques throughout the internship tenure.

I am immensely grateful to the leadership and technical staff of the **Information Network Security Administration (INSA)** of Ethiopia for granting me the rare privilege to serve as a Software Engineering Intern within their specialized cybersecurity and software development directorate.

My profound appreciation goes to my industrial supervisor at INSA, whose technical mentorship, patient code reviews, deep architectural insights, and high standards for software security profoundly elevated my software craftsmanship. His willingness to discuss complex topics—from OAuth2/OIDC token exchanges and XMPP BOSH protocol handshakes to PostgreSQL indexing strategies—greatly broadened my professional horizons.

I also express heartfelt gratitude to the entire INSA engineering team and fellow colleagues for welcoming me into their collaborative agile environment, offering continuous assistance, and sharing their valuable industry experiences.

Finally, I dedicate my warmest thanks to my family and friends for their unconditional love, moral encouragement, emotional support, and understanding during intensive project delivery phases.

LIST OF ACRONYMS

Acronym	Full Meaning	Acronym	Full Meaning
ABAC	Attribute-Based Access Control	**NFR**	Non-Functional Requirement
API	Application Programming Interface	**OIDC**	OpenID Connect
AOP	Aspect-Oriented Programming	**ORM**	Object-Relational Mapping
KMS	Knowledge Management System	**OS**	Operating System
BOSH	Bidirectional-streams Over Synchronous HTTP	**PKI**	Public Key Infrastructure
CI/CD	Continuous Integration / Deployment	**RAG**	Retrieval-Augmented Generation
CII	Critical Information Infrastructure	**RBAC**	Role-Based Access Control
CIIP	Critical Information Infrastructure Protection	**REST**	Representational State Transfer
CPU	Central Processing Unit	**RFC**	Request For Comments
CSE	Computer Science and Engineering	**SASL**	Simple Authentication and Security Layer
CSS	Cascading Style Sheets	**SDLC**	Software Development Life Cycle
DBMS	Database Management System	**SHA**	Secure Hash Algorithm
DDL	Data Definition Language	**JWT**	JSON Web Token
DTO	Data Transfer Object	**SOP**	Standard Operating Procedure
ERD	Entity Relationship Diagram	**SQL**	Structured Query Language
FTS	Full-Text Search	**SSO**	Single Sign-On
GIN	Generalized Inverted Index	**TLS**	Transport Layer Security
GRC	Governance, Risk, and Compliance	**MVC**	Model-View-Controller
HTML	HyperText Markup Language	**URI**	Uniform Resource Identifier
JPA	Java Persistence API	**URL**	Uniform Resource Locator
HTTPS	HyperText Transfer Protocol Secure	**UUID**	Universally Unique Identifier
IDE	Integrated Development Environment	**UX**	User Experience
INSA	Information Network Security Administration	**XMPP**	Extensible Messaging and Presence Protocol
IoC	Inversion of Control	**JWKS**	JSON Web Key Set

Contents

CHAPTER 1: INTRODUCTION 1

1.1 General Overview About the Internship 1

1.2 Background of the Hosting Organization 2

CHAPTER 2: MANAGEMENT, TRAINING, AND EMPLOYEE INFORMATION 3

2.1 Major Duty Assigned to Perform 3

2.2 How Did You Become Oriented with the Responsibilities You Were Assigned? 4

2.3 How Did Your Supervisor Help You to Succeed in the Assignment You Were Given? 5

2.4 What Qualities Did You Develop Which Allowed You to Succeed in Your Daily Duties? 7

CHAPTER 3: SPECIFIC JOB INFORMATION 8

3.1 Daily Technical Duties and Responsibilities 8

3.2 Other Responsibilities During the Internship 15

3.3 Technical Knowledge and Skills from Coursework Beneficial for Assignment 16

CHAPTER 4: REFLECTION AND CONCLUSION 17

4.1 How Did This Internship Fit Your Career Goals? 17

4.2 Did Your Career Goals Change as a Result of This Internship Experience? 17

4.3 Discuss Your Feelings About the Value of This Internship 18

4.4 What Challenges Have You Faced During the Internship? 18

4.5 Discuss Your Strengths and Areas for Improvement as Self-Evaluation 20

4.6 Conclusion 21

CHAPTER 5: RECOMMENDATIONS 21

5.1 Personal Opinion of the Company and the Internship 21

5.2 Recommendations for Improving the Internship Program 22

REFERENCES / BIBLIOGRAPHY 24

CHAPTER 1: INTRODUCTION

1.1 General Overview About the Internship

Industrial internship training is a vital cornerstone of the engineering curriculum at **Adama Science and Technology University (ASTU)** under the **School of Electrical Engineering and Computing (SoEEC)**. The program bridges classroom theoretical instruction with real-world, production-grade enterprise software engineering.

During this two-month internship (45 working days, July 07 – September 04, 2026) at the **Information Network Security Administration (INSA)** in Addis Ababa, Ethiopia, I was assigned to the Software R&D and Cybersecurity Infrastructure Directorate. My primary duty was the design, implementation, and security hardening of the **Enterprise Knowledge Management System (INSA-KMS)** a sovereign digital repository, full-text search engine, and secure real-time collaboration ecosystem designed to safeguard institutional memory across INSA directorates.

1.1.1 Objectives of the Internship

The internship encompassed three complementary dimensions:

1. **Academic Objectives:** Apply university computer science principles—object-oriented design, relational database normalization (3NF), software engineering lifecycles, and cryptographic algorithms—to production software engineering under enterprise constraints.
2. **Professional Objectives:** Cultivate professional engineering ethics, agile teamwork (daily standups, sprint planning, peer code reviews), and strict adherence to national cybersecurity compliance standards.
3. **Technical Objectives:**
    - Architect a modular 3-tier enterprise Knowledge Management System;
    - Implement federated OpenID Connect (OIDC) authentication using **Keycloak 26.7.2** with asymmetric RS256 JWT validation;
    - Build an asynchronous document ingestion pipeline using **Apache Tika** and **PostgreSQL 15 Full-Text Search (FTS)** with Generalized Inverted Indexes (GIN);
    - Integrate an on-premise, encrypted **Jitsi Meet / Prosody XMPP** video collaboration subsystem with fail-closed HMAC-SHA256 token room admission;
    - Establish tamper-evident audit logging via Spring Aspect-Oriented Programming (AOP).

1.1.2 Scope of Work Expected

The expected scope spanned the full software development lifecycle (SDLC):

- **Requirements Engineering:** Translating INSA document governance policies and data classifications (PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED) into formal functional specifications.
- **Backend Engineering:** Developing modular Spring Boot 3.3 microservices with Spring Security OAuth2 resource servers, transactional JPA entities, and RESTful APIs.
- **Frontend Development:** Engineering an accessible, responsive Next.js 14 (React 18, TypeScript, Tailwind CSS) client with embedded PDF.js previewing and interactive multi-user selection.
- **Security & Testing:** Authoring automated JUnit 5 test suites, executing live runtime integration verifications, and orchestrating multi-container Docker Compose environments.

1.1.3 Outputs Expected

The primary tangible outputs delivered upon completion included:

- **D-1: Hardened Backend API:** Spring Boot 3.3 RESTful engine with OAuth2 resource server validation.
- **D-2: Next.js 14 Web Portal:** Responsive client with document explorer and in-browser PDF canvas previewer.
- **D-3: Keycloak Identity Realm:** Configured kms-realm with OIDC clients, custom claims, and 7 RBAC roles.
- **D-4: Full-Text Search Engine:** Asynchronous Apache Tika text extractor paired with PostgreSQL 15 GIN indexing.
- **D-5: Secure Video Collaboration:** Self-hosted Jitsi Meet/Prosody with HMAC-SHA256 token admission.
- **D-6: Tamper-Evident Audit Logging:** Spring AOP aspect capturing immutable, forensic-grade audit trails.
- **D-7: Verification Test Suites:** 20/20 passing JUnit integration tests and live PowerShell runtime verifications.

1.2 Background of the Hosting Organization

1.2.1 Description of INSA

The **Information Network Security Administration (INSA)** is Ethiopia's premier federal cybersecurity, technical intelligence, and cyber defense agency. Established by Council of Ministers Regulation No. 130/2006 and re-established with expanded statutory authority under Proclamation No. 808/2013, INSA is legally mandated to protect Ethiopia's information and critical information infrastructure (CII), ensure national cyber sovereignty, and foster sovereign technological capabilities.

Headquartered on Welo Sefer China Street in Addis Ababa, INSA operates as a high-technology state institution defending national critical infrastructures—telecommunications, financial transaction networks, electrical grids, aviation systems, and government e-services—against cyber warfare, espionage, unauthorized intrusion, and digital sabotage.

INSA is structured into specialized directorates:

- *Cybersecurity Infrastructure & Operations Directorate (including EthioCERT);*
- *Cryptographic Engineering & Public Key Infrastructure (PKI) Directorate;*
- *Software Research & Development and Systems Engineering Directorate;*
- *Governance, Risk, and Compliance (GRC) & Cyber Auditing Directorate;*
- *Digital Forensics & Cyber Crime Investigation Directorate.*

1.2.2 Mission, Vision, and Core Values of INSA

Vision:

"To realize a secure, resilient, and sovereign Ethiopian cyberspace that empowers sustainable national transformation and technological advancement."

Mission:

"To ensure national cyber security by defending information and information infrastructure, building advanced cryptographic and cybersecurity capabilities, conducting technical intelligence, and fostering sovereign digital trust."

Core Values:

- **National Interest Above All:** Unwavering dedication to safeguarding Ethiopia's digital sovereignty.
- **Innovation and Excellence:** Continuous mastery of cutting-edge technologies and sovereign research.
- **Integrity and Confidentiality:** Uncompromising professional ethics, trust, and secrecy.
- **Vigilance and Proactiveness:** Continuous cyber monitoring, threat hunting, and rapid response.
- **Collaboration and Synergy:** Inter-disciplinary teamwork and institutional knowledge transfer

1.2.3 Products and Services of INSA

INSA delivers critical technical services and sovereign digital products:

1. **National Incident Response (EthioCERT):** 24/7 surveillance of national cyberspace, early warning advisories, and emergency cyber incident containment.
2. **Cybersecurity Audits & System Inspection:** Comprehensive penetration testing, vulnerability assessments, and compliance audits against ISO/IEC 27001 and NIST SP 800-53 standards.
3. **Cryptographic Engineering & PKI:** National root certification authority administration, digital signature frameworks, and hardware security module (HSM) deployment.
4. **Digital Forensics & Investigation:** Forensic acquisition, court-admissible evidence preservation, and deep analysis of compromised IT environments.
5. **Secure Software Development:** Hardened enterprise platforms and sovereign defense applications, including the **INSA-KMS** platform engineered during this internship.
6. **Governance, Risk, & Compliance (GRC):** National baseline cybersecurity directives, institutional risk assessments, and specialized capacity-building training.

CHAPTER 2: MANAGEMENT, TRAINING, AND EMPLOYEE INFORMATION

2.1 Major Duty Assigned to Perform

As a Software Engineering Intern within the Software Research & Development and Cybersecurity Infrastructure Directorate at the **Information Network Security Administration (INSA)**, my primary operational duty was the full-stack engineering, security architecture design, integration, and verification of the **INSA Knowledge Management System (INSA-KMS)**.

Prior to this engineering initiative, institutional knowledge across INSA's multidisciplinary divisions—including vulnerability advisories, penetration testing methodology manuals, internal cryptographic documentation, research findings, standard operating procedures, and compliance policies—was fragmented across isolated network drives, personal workstations, email threads, and disparate messaging platforms. This uncoordinated landscape resulted in significant institutional vulnerabilities:

- Critical technical knowledge was susceptible to permanent loss upon personnel turnover or reassignment;
- Parallel teams frequently duplicated complex analytical research due to zero cross-directorate discovery mechanisms;
- Access to classified documents lacked uniform, centralized governance, making verification against national clearance levels challenging;

- Audit trails for document access, modification, and dissemination were non-existent or fragmented across disparate server logs.

To systematically eliminate these institutional bottlenecks, my major duty was formulated as follows:

1. **Lead the Full-Stack Engineering of INSA-KMS:** Design, develop, and test an enterprise-grade web application platform that centralizes organizational documents, provides sub-second semantic and full-text search, enforces granular access control, and integrates virtual video collaboration.
2. **Implement Federated Identity and Security Architecture:** Configure and integrate an on-premise **Keycloak 26.7.2** identity broker operating under OpenID Connect (OIDC) and OAuth 2.0 specificat3ions. The system had to validate cryptographically signed RS256 JSON Web Tokens (JWT) on every incoming API request and enforce fine-grained Role-Based Access Control (RBAC) across seven distinct enterprise roles.
3. **Develop Asynchronous Document Ingestion and Full-Text Search Engine:** Engineer a resilient document processing pipeline that accepts heterogeneous file formats (PDF, DOCX, XLSX, TXT), computes SHA-256 cryptographic hashes for tamper detection, extracts textual content asynchronously using **Apache Tika**, and persists structured tokens into **PostgreSQL 15** utilizing tsvector data types and Generalized Inverted Index (GIN) structures.
4. **Engineer a Fail-Closed Virtual Video Discussion Subsystem:** Construct an end-to-end encrypted virtual video conferencing and collaboration capability utilizing self-hosted **Jitsi Meet / Prosody XMPP**. The subsystem required the generation and validation of short-lived HMAC-SHA256 JWT tokens, debounced multi-user participant selectors, host-participant role segregation, and fail-closed room isolation to guarantee that unauthorized users cannot eavesdrop on sensitive discussions.
5. **Establish Tamper-Evident Governance and Audit Logging:** Implement an aspect-oriented logging engine using Spring AOP that captures immutable, forensic-grade audit records for every state mutation across the enterprise.

2.2 How Did You Become Oriented with the Responsibilities You Were Assigned?

Navigating the transition from an academic university environment to an elite national cyber defense agency requires a structured, security-conscious orientation and onboarding regimen. At INSA, orientation was executed across three comprehensive stages:

Stage 1: Security Clearances, Non-Disclosure Agreements, and Facility Protocols

Upon arrival at INSA headquarters, the initial orientation phase was administered by the Human Resources Directorate in coordination with Internal Security. Given the sensitive nature of INSA's national security mandate, all interns underwent comprehensive identity verification, completed formal background checks, and signed legally binding Non-Disclosure Agreements (NDAs). This stage encompassed briefings on:

- **Physical and Facility Security:** Badge-controlled zone access, biometric checkpoints, clean desk policies, and restrictions regarding personal recording devices and unauthorized external storage media in development areas.

- **Cyber Hygiene and Information Classification:** Rigorous adherence to the national data classification hierarchy (PUBLIC, INTERNAL, CONFIDENTIAL, and RESTRICTED), understanding that handling classified system codebases demands air-gapped development environments and strictly monitored network perimeters.

Stage 2: Directorate and Team Introduction

Following formal security clearance, I was escorted to the Software Research & Development and Systems Engineering Directorate, where I was formally introduced to the Directorate Director, the Systems Architect, my designated Industrial Supervisor, and the development team.

- During this orientation session, the team lead presented the high-level roadmap of the agency's digital transformation initiatives and positioned the INSA-KMS project within the broader organizational strategy.
- I was introduced to the team's collaborative engineering culture, which adheres strictly to Agile Scrum methodologies, two-week sprint iterations, daily standup check-ins, and peer-reviewed pull request workflows.

Stage 3: Technical Environment Provisioning and Codebase Orientation

The final onboarding phase focused on technical immersion:

- **Development Workstation Setup:** Provisioning a hardened corporate development workstation running under localized security policies. I configured local development dependencies, including OpenJDK 21 (Temurin), Apache Maven 3.8+, Node.js 18 LTS, Docker Desktop, PostgreSQL 15, and the Keycloak 26.7.2 identity server.
- **Architecture and Requirements Walkthrough:** My supervisor conducted an in-depth architectural briefing using the *KMS Requirements Specification* document. We walked through the entity-relationship models, API contracts, Keycloak realm definitions (kms-realm.json), and the Jitsi Meet container orchestration configurations.
- **Hands-On Starter Task:** To validate environment readiness and familiarize myself with the codebase, my supervisor assigned an initial warm-up task: setting up the local Keycloak container, importing the realm seed configuration via PowerShell scripts, and executing the automated Spring Boot test suite. This immediate hands-on exercise instilled confidence and provided immediate clarity regarding the system's operational topology.

2.3 How Did Your Supervisor Help You to Succeed in the Assignment You Were Given?

The guidance, technical stewardship, and supportive leadership provided by my industrial supervisor were instrumental in transforming complex, high-stakes project requirements into a structured, highly successful engineering endeavor. My supervisor fostered an environment of high technical expectations balanced by continuous pedagogical support. His assistance manifested across several vital dimensions:

1. Architectural Guidance and Technical Direction

Whenever architectural forks or technical ambiguities arose, my supervisor provided deep engineering counsel. For example, during the initial design of the virtual video collaboration subsystem, there was debate over whether to rely on an external commercial cloud meeting API or construct a fully self-hosted, sovereign WebRTC infrastructure. My supervisor reinforced INSA's sovereign security mandate, directing me to containerize Jitsi Meet and Prosody XMPP locally. He guided me through the mathematical and cryptographic underpinnings of RFC 7519 HMAC-SHA256 token generation,

ensuring that Jitsi room tokens carried strict audience (aud), issuer (iss), and expiration (exp) claims that matched Prosody's BOSH authentication module.

2. Systematic Code Reviews and Engineering Rigor

My supervisor instituted a rigorous code review process for every pull request I authored. Rather than simply evaluating functional correctness, his reviews evaluated:

- **Defensive Coding Practices:** Enforcing thorough input validation, parameterized JPA queries to eliminate SQL injection risks, and sanitization of user-supplied HTML in discussion threads to prevent Cross-Site Scripting (XSS).
- **Architectural Separation of Concerns:** Ensuring that controller layers remained thin, business logic was encapsulated within transactional services, and data transfer objects (DTOs) decoupled internal database schemas from external REST API representations.
- **Security Invariants:** Verifying that authorization boundaries failed closed. For instance, when designing the participant invitation endpoints, he insisted that client-submitted user IDs must be cross-referenced against active database records rather than blindly trusted from incoming request bodies.

3. Structured Mentorship Cadence and Problem-Solving

My supervisor maintained a disciplined communication cadence throughout the 45 working days, combining daily morning standups with dedicated weekly architectural review sessions:

Table 2.1: Weekly Supervisory Mentorship and Review Cadence

Day / Time	Session Type	Focus and Deliverables Reviewed
Daily (08:30 – 08:50 AM)	Agile Morning Standup	Review of previous day's completed tasks, identification of technical blockers, and daily target setting.
Wednesday (02:00 – 03:30 PM)	Deep-Dive Architecture Review	Code walkthroughs, security vulnerability scanning, database query optimization, and refactoring sessions.
Friday (03:30 – 05:00 PM)	Sprint Retrospective & Demo	Live demonstration of functional features, peer review feedback, and sprint milestone validation.

4. Resource Provisioning and Professional Empowerment

My supervisor ensured that I had unrestricted access to technical documentation, enterprise reference architectures, and necessary compute infrastructure. Furthermore, he empowered me to make autonomous architectural decisions regarding frontend component design (e.g., implementing debounced multi-user search with Radix UI primitives) and backend optimization (e.g., GIN index tuning in PostgreSQL), treating me as a trusted, capable engineering colleague rather than merely a temporary intern.

2.4 What Qualities Did You Develop Which Allowed You to Succeed in Your Daily Duties?

The rigorous, mission-focused environment at INSA accelerated my personal, professional, and technical maturity. Over the course of the two-month internship, I actively cultivated several core qualities that were essential to succeeding in my daily engineering responsibilities:

1. Professional Resilience and Methodological Debugging

Enterprise software development frequently involves complex integration issues where bugs span across multiple distributed systems such as browser CORS policies, Keycloak token expiration, Spring Security filter chains, and PostgreSQL connection pooling. I developed deep professional resilience and a methodical, hypothesis-driven approach to debugging. Instead of resorting to trial-and-error guessing, I learned to isolate variables systematically: inspecting HTTP request and response headers using network proxies, analyzing Spring Boot debug logs, decoding JWT payloads using cryptographic tools, and monitoring database transaction states. This disciplined mindset allowed me to resolve subtle token synchronization issues between Keycloak and Prosody without panic or frustration.

2. Defensive Engineering and Security-First Mindset

Prior to the internship, academic assignments often prioritized "happy path" functional correctness—verifying whether an algorithm works under ideal inputs. Immersed within INSA's cybersecurity culture, I internalized a relentless, defensive engineering philosophy:

- Always assume inputs are potentially malicious until explicitly validated and sanitized;
- Enforce the Principle of Least Privilege across all database accounts, API routes, and container processes;
- Ensure that every security boundary fails closed—if an authentication token is malformed, expired, or tampered with, the system must immediately terminate the operation with a clean, uninformative error message while recording a detailed forensic audit log in the background.

3. Autonomy and Proactive Technical Inquiry

While supervisory support was always available, the fast-paced delivery schedule demanded high levels of self-reliance. I developed the ability to parse complex, dense technical documentation independently including the official Spring Security 6 OAuth2 specifications, Keycloak REST Admin API schemas, Jitsi Prosody Lua authentication modules, and RFC standards (RFC 6749, RFC 7519, RFC 6120). When faced with an unfamiliar technical challenge, I proactively researched solutions, constructed isolated proof-of-concept scratch scripts, benchmarked alternatives, and presented well-reasoned recommendations to my supervisor.

4. Clear Technical Communication and Collaboration

Working within a multidisciplinary team required the ability to articulate technical concepts with clarity, precision, and conciseness. I learned to communicate effectively across diverse technical domains: explaining database indexing trade-offs to backend developers, discussing API response payload contracts with frontend engineers, and summarizing system security posture for compliance officers. Furthermore, I developed a strong appreciation for comprehensive documentation, maintaining clean architectural blueprints, inline code documentation, and reproducible deployment scripts that ensure long-term system maintainability.

5. Punctuality, Discipline, and Time Management
Balancing the delivery of multiple interconnected subsystems authentication, document storage, full-text search, video conferencing, and audit logging within a tight 45-day schedule required exceptional time management. I cultivated strict personal discipline, arriving punctually at 08:00 AM every working day, breaking large sprint goals into manageable daily deliverables, utilizing issue tracking boards effectively, and consistently meeting or exceeding sprint deadlines.

CHAPTER 3: SPECIFIC JOB INFORMATION

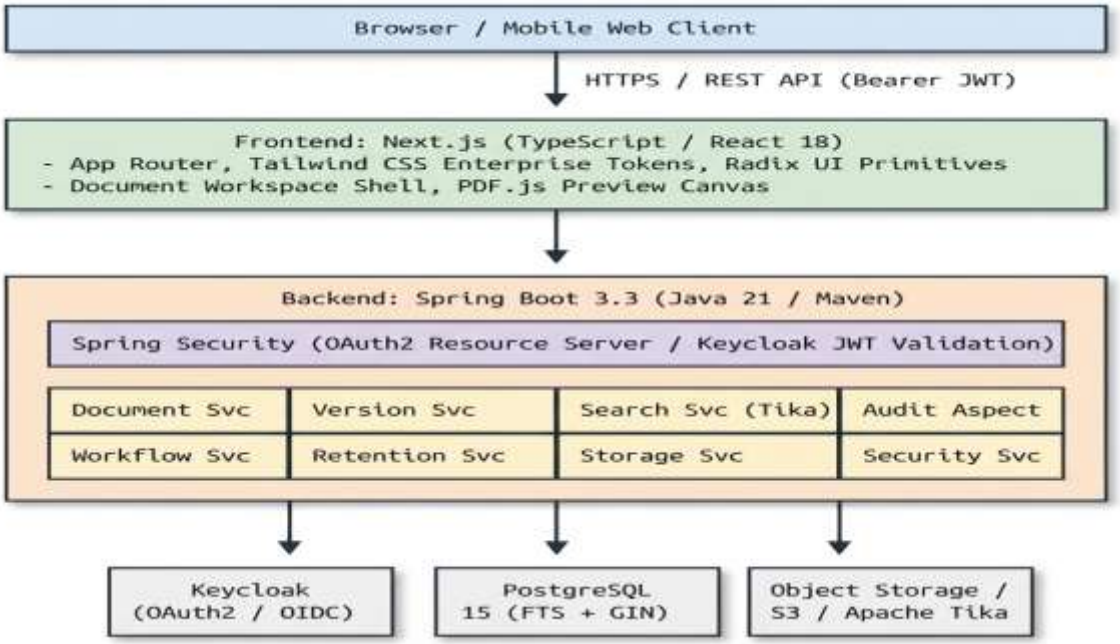
3.1 Daily Technical Duties and Responsibilities

During the two-month internship tenure at the **Information Network Security Administration (INSA)**, my primary daily technical duty was the end-to-end engineering, hardening, integration, and empirical verification of the **Enterprise Knowledge Management System (INSA-KMS)**. This section provides an in-depth technical analysis of the problem context, system topology, core architectural subsystems, software engineering workflows, and empirical verification results.

3.1.1 Problem Statement and Context of INSA-KMS

As Ethiopia's central cybersecurity and technical intelligence agency, INSA processes vast volumes of highly classified and technically complex information daily. Across its directorates—such as EthioCERT, Cryptographic Engineering, Forensics, and Critical Information Infrastructure Protection (CIIP)—personnel continuously author threat intelligence bulletins, vulnerability assessments, penetration testing methodologies, software design blueprints, incident response playbooks, and regulatory frameworks.
Historically, this intellectual capital was stored in an uncoordinated, fragmented manner:

1. **Siloed Storage Repositories:** Documents were scattered across local employee hard drives, shared Windows network folders (SMB/CIFS), private cloud accounts, and email attachments. When specialized engineers were reassigned or exited the organization, critical operational knowledge was frequently lost or rendered inaccessible.
2. **Lack of Centralized Discovery:** Locating a specific cyber vulnerability whitepaper or historical incident report often required days of manual inquiries across different departments, resulting in significant duplication of complex analytical and reverse-engineering efforts.
3. **Absence of Unified Access Governance:** File access was governed by crude operating system file permissions rather than an enterprise-wide, fine-grained access control model. Enforcing national information clearance levels
(PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED) across digital files was technically infeasible under the legacy system.
4. **Security Risks and Lack of Auditability:** There was no cryptographically verifiable audit trail of who created, accessed, modified, downloaded, or shared classified documents. In the event of an internal security breach or data leak, forensic investigators had no reliable, centralized access log to examine.



5. **Disconnected Collaboration Channels:** Collaboration around sensitive technical documents took place over unencrypted or external communication tools, exposing confidential discussions to potential surveillance and data interception.

To eliminate these vulnerabilities, INSA initiated the **INSA-KMS** project. The mandate was to engineer an air-gapped, sovereign, enterprise-grade Knowledge Management System that acts as a single, governed, highly searchable, and tamper-evident source of truth for all internal organizational knowledge assets.

Figure 3.1: High-Level 3-Tier Enterprise Architecture of INSA-KMS

3.1.2 Architectural Framework and System Topology

To achieve high availability, strict security boundaries, and modular maintainability, INSA-KMS was designed around a resilient 3-tier enterprise architecture. The system decouples presentation, business services, identity federation, and data persistence into cleanly separated layers communicating over secure, standard protocols.

Table 3.1: Technology Stack Matrix for INSA-KMS

Architectural Tier	Technology / Framework	Version	Engineering Justification
Frontend UI/UX	Next.js (React, TypeScript)	14.2+	Server-Side Rendering (SSR) and Client-Side Hydration via App Router; static route optimization and strict type safety.
Styling & Components	Tailwind CSS + Radix UI	3.4+ / 1.x	Headless, accessible primitives with custom enterprise cyber design system tokens; zero runtime CSS overhead.
Backend Framework	Spring Boot (Java 21 LTS)	3.3.3	Enterprise-grade dependency injection, high-throughput virtual threads (Project Loom), and comprehensive security ecosystem.
Security & Identity	Keycloak (OIDC / OAuth 2.0)	26.7.2	Sovereign, open-source identity provider; centralized credential management, SSO, brute-force protection, and RS256 token signing.
Relational Database	PostgreSQL Server	15.6	ACID-compliant relational data modeling; native tsvector and tsquery full-text search engine with Generalized Inverted Indexes (GIN).

Document Ingestion	Apache Tika Core & Parsers	2.9.2	Asynchronous metadata extraction and full-text content parsing across PDF, DOCX, XLSX, ODT, and plain-text files.
Video Conferencing	Jitsi Meet & Prosody XMPP	Latest Stable	Containerized sovereign WebRTC conferencing; XMPP BOSH signaling hardened via HMAC-SHA256 JWT admission tokens.
Containerization	Docker & Docker Compose	26.x	Deterministic, multi-stage microservices packaging, environment parity, and isolated network bridge boundaries.

3.1.3 Subsystem 1: Identity and Access Management (Keycloak OIDC Federation)

In high-security environments, storing user credentials, password hashes, and session state within the application database introduces severe attack surfaces. In accordance with INSA's sovereign security guidelines, all authentication and identity management operations were offloaded to an enterprise **Keycloak 26.7.2** identity broker operating under the **OpenID Connect (OIDC)** and **OAuth 2.0** specifications.

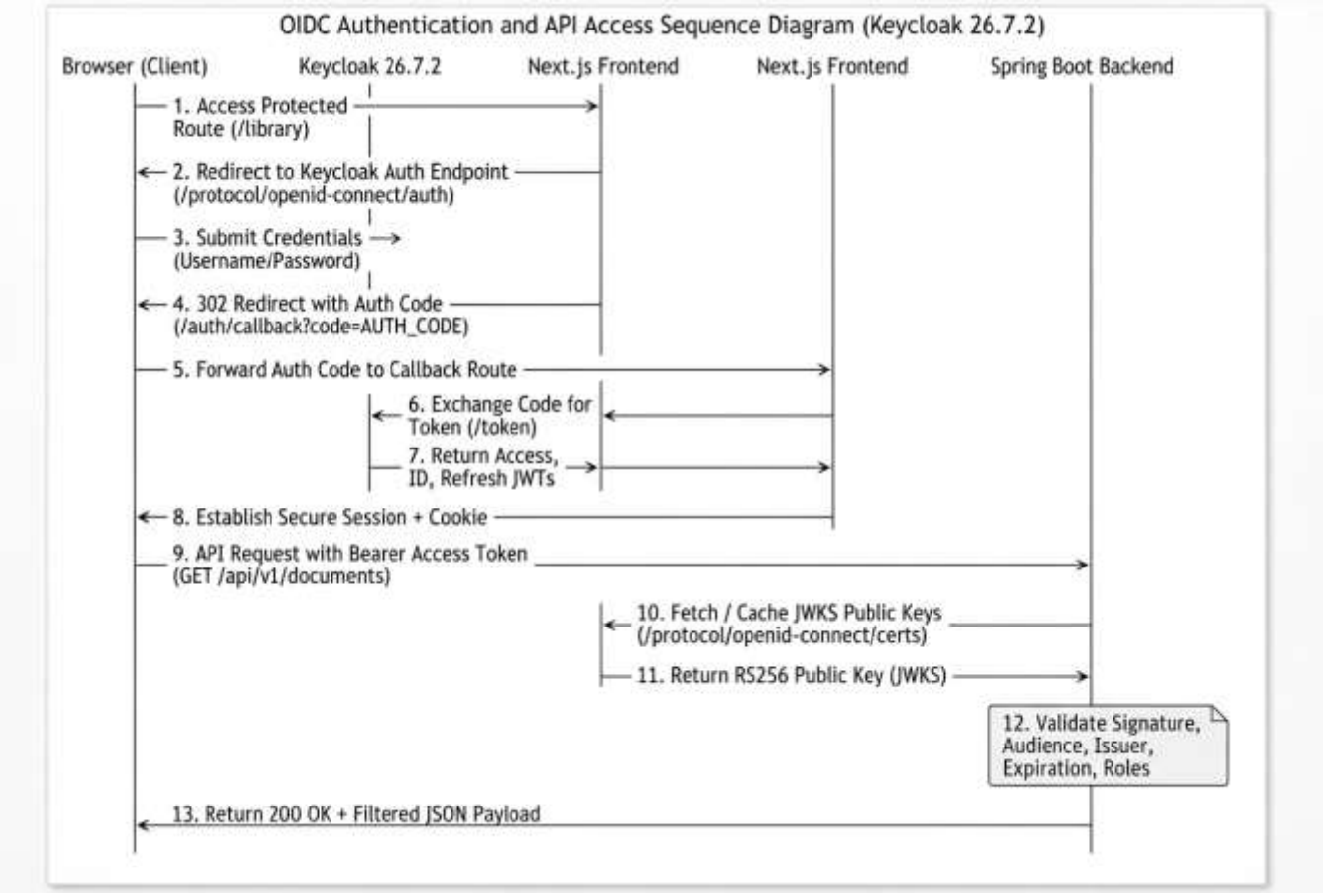


Figure 3.2: OpenID Connect (OIDC) Authorization Code Flow with Keycloak 26

Technical Implementation Details:

1. **Dedicated Enterprise Realm:** A dedicated realm named kms-realm was constructed (exported as keycloak/kms-realm.json). Within this realm, an OpenID Connect client named kms-frontend-client was provisioned with standard flow enabled and public client credentials, restricted strictly to authorized redirect URIs (http://localhost:3000/*, https://kms.insa.gov.et/*).
2. **Cryptographic Token Verification:** The Spring Boot backend acts strictly as an **OAuth2 Resource Server** (spring-boot-starter-oauth2-resource-server). It never handles raw user passwords. Upon receiving an incoming HTTP request containing an Authorization: Bearer <JWT> header, Spring Security's JwtDecoder verifies:
    - **Cryptographic Signature:** The token's signature is verified against Keycloak's JSON Web Key Set (JWKS) public certificates endpoint (/realms/kms-realm/protocol/openid-connect/certs) utilizing asymmetric RS256 (RSA Signature with SHA-256).
    - **Issuer and Audience Invariants:** The iss claim must match http://localhost:8080/realms/kms-realm, and the token must not be expired (exp > now).
    - **Role Extraction:** A custom KeycloakJwtGrantedAuthoritiesConverter was authored in Java to extract roles nested inside the realm_access.roles JSON claim and translate them into Spring Security GrantedAuthority objects prefixed with ROLE_.

3.1.4 Subsystem 2: Document Repository and Lifecycle Management

The core operational function of INSA-KMS is providing a robust, version-controlled repository for internal digital assets.

1. Entity Modeling and Metadata Taxonomy

Every uploaded document is represented in the database by two primary entities:

- **Document Entity:** Represents the logical document entity. It captures immutable business metadata: unique UUID (id), title, description, category, folder hierarchy, department ownership, security classification (PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED), retention policy, and current active version pointer.
- **DocumentVersion Entity:** Represents an immutable physical snapshot of the file. Every time an existing document is updated, a new DocumentVersion row is created, capturing: version number (e.g., v1.0, v1.1), file name, MIME type, byte size, storage URI path, uploader UUID, upload timestamp, change summary, and a SHA-256 cryptographic checksum.

2. Cryptographic Integrity and Anti-Tampering (SHA-256)

To detect unauthorized modification or bit rot at the storage layer, the document ingestion pipeline computes the cryptographic SHA-256 hash of the incoming byte stream immediately upon upload: Checksum = SHA-256(InputStream) This hexadecimal digest is permanently recorded in document_versions.checksum_sha256. Prior to streaming any file download to an authorized client, the storage service recomputes the checksum of the stored file on disk and verifies it against the recorded database value. If a mismatch occurs, the download is aborted with an alert indicating file tampering.

3. Concurrency Control and Collaborative Check-in / Check-out

To prevent conflicting overwrites when multiple engineers collaborate on the same standard operating procedure or policy document, a pessimistic check-in/check-out lock mechanism was engineered:

- When an authorized Contributor initiates an edit, the document is flagged as is_locked = true with locked_by_user_id = <UUID> and locked_at = <Timestamp>.
- While locked, other users can view and download the latest published version but are prevented from uploading revisions.
- Upon completing edits, the locking user uploads the updated file, releasing the lock and automatically incrementing the version index. Administrators and Content Owners possess administrative override capabilities to break stale locks with mandatory audit logging.

3.1.5 Subsystem 3: Asynchronous Full-Text Search and Apache Tika Indexing

In an enterprise repository storing thousands of technical documents, hierarchical folder browsing alone is insufficient for rapid information retrieval. INSA-KMS implements an intelligent full-text search (FTS) engine powered by an asynchronous **Apache Tika** text extraction pipeline coupled with **PostgreSQL 15 GIN** indexes.

Engineering Pipeline:

1. **Asynchronous Decoupling:** Parsing large multi-megabyte technical manuals or PDF specifications during an HTTP POST upload request causes unacceptable request timeouts and degrades server throughput. The extraction was decoupled using Spring's @Async thread pool (ThreadPoolTaskExecutor).
2. **Multi-Format Extraction:** The background worker invokes **Apache Tika Core** (AutoDetectParser), which inspects MIME magic bytes and extracts raw text from PDF, DOCX, XLSX, PPTX, ODT, HTML, and TXT files.
3. **PostgreSQL Lexical Analysis and GIN Indexing:**
   The extracted text is persisted into a dedicated search_vector column typed as tsvector. PostgreSQL's native lexical analyzer parses the text into normalized tokens (lexemes), strips stop-words, and applies linguistic stemming (e.g., transforming "vulnerabilities", "vulnerable", and "vulnerability" to the common root vulner). A Generalized Inverted Index (GIN) was created across the table:
   CREATE INDEX idx_document_search_vector ON documents USING GIN(search_vector);
4. **Search Execution and Relevancy Ranking:**
   When a user inputs a query (e.g., "cryptographic key exchange vulnerability"), the API transforms the string into a formatted tsquery executing the query:

```
SELECT id, title, category, classification,
    ts_rank_cd(search_vector, query) AS rank,
    ts_headline('english', extracted_text, query, 'StartSel=<b>, StopSel=</b>,
MaxWords=35, MinWords=15') AS snippet
FROM documents, to_tsquery('english', 'cryptographic & key & exchange &
vulnerability') query
WHERE search_vector @@ query
  AND classification IN (:allowedClassifications)
ORDER BY rank DESC;
```

This architecture delivers sub-second search responses across the entire repository while highlighting matching keywords directly in search result snippets.

3.1.6 Subsystem 4: Role-Based and Attribute-Based Access Control (RBAC & ABAC)

Given the national security context of INSA, data confidentiality is governed through a dual-layered security model combining **Role-Based Access Control (RBAC)** for operational privileges with **Attribute-Based Access Control (ABAC)** for data classification clearance.

1. Role-Based Access Control (RBAC) Matrix

Seven distinct enterprise roles are configured within the Keycloak realm and mapped to Spring Security authority tokens:

Table 3.2: Keycloak Enterprise Realm Roles and Authorization Permissions

Realm Role	Scope and Responsibilities	Specific System Permissions
ROLE_SUPER_ADMIN	Global System Administration	Manage system configuration, database backups, realm settings, and full system override.
ROLE_ADMIN	Operational Administration	User lifecycle management, department provisioning, storage quota allocation, and system telemetry.
ROLE_CONTENT_OWNER	Departmental Knowledge Lead	Create folder hierarchies, assign departmental permissions, approve document publication, and archive content.
ROLE_CONTRIBUTOR	Technical Staff / Engineer	Upload new documents, author new versions, check-out files for editing, participate in discussions.
ROLE_VIEWER	Read-Only Consumer	Search, view, and download documents within authorized classification tiers; no modification rights.
ROLE_COMPLIANCE_OFFICER	Governance & Legal Auditor	Define retention schedules, inspect legal holds, review access logs, and generate regulatory compliance reports.
ROLE_IT_SECURITY	Cybersecurity Monitoring	Real-time inspection of forensic audit logs, monitoring authentication anomalies, and investigating security alerts.

2. Attribute-Based Access Control (ABAC) Classification Clearance

In addition to their role, every user possesses a security clearance attribute (clearance_level). Documents are assigned one of four classification levels:

Table 3.3: Document Security Classification Matrix (ABAC Clearance)

Classification Level	Definition and Sensitivity	Access Clearance Required	Handling Controls
PUBLIC	Information approved for public dissemination.	Any authenticated user.	Standard storage; watermarking disabled.
INTERNAL	Routine internal operational memos and technical guides.	ROLE_VIEWER or higher with INTERNAL clearance.	Restricted to INSA internal network IP ranges.
CONFIDENTIAL	Proprietary research, network topologies, vulnerability reports.	ROLE_CONTRIBUTOR or higher with CONFIDENTIAL clearance.	Strict audit logging on every read; download watermarked with user ID.
RESTRICTED	High-level cryptographic keys, active cyber-incident investigations.	Explicit per-user ACL or SUPER_ADMIN with RESTRICTED clearance.	Two-factor re-authentication required prior to access; downloads blocked.

Every database search and document fetch query enforces these bounds at the repository layer using Spring Data JPA specifications: Access Granted $\iff$ (User Role $\geq$ Required Action Privilege) $\wedge$ (User Clearance $\geq$ Document Classification)

3.1.7 Subsystem 5: Secure Virtual Video Discussion and Jitsi Meet Integration

To facilitate secure real-time collaboration among engineers analyzing sensitive cyber incidents, a dedicated **Virtual Video Discussion Subsystem** was architected, replacing third-party commercial meeting software with an on-premise, self-hosted **Jitsi Meet / Prosody XMPP** cluster.

Key Engineering Invariants Enforced:

1. **Interactive Debounced Participant Selector:**
   The frontend modal (CreateVideoSessionModal.tsx) replaces static text inputs with an interactive user selector. When opened, it automatically loads active KMS colleagues, transitioning dynamically to debounced search queries across full name, username, and email. Removable chips, keyboard navigation (ArrowDown, ArrowUp, Enter, Escape), and host self-selection disabling ("You are the host") ensure a seamless user experience.
2. **Fail-Closed Room Isolation:**
   Uninvited users attempting to access a video session receive a 403 Forbidden error and are never issued a room token.
3. **Cryptographic Token Binding (RFC 7519):**
   Tokens are signed using HMAC-SHA256 with a 300-second time-to-live (TTL). The token payload explicitly binds the user to a single room name: Payload = iss:"kms-backend",aud:"jitsi",room:"kms-room-" + sessionUUID,context:user: ...
4. **Zero URL Parameter Leakage:**
   Meeting links are returned as clean URLs (/meeting/<uuid>), passing the admission token securely through authenticated session payloads to prevent tokens from leaking into browser history, proxies, or server access logs.

Implementation Structure:

1. **Custom Annotation (@AuditLog):**
   A Java annotation @AuditLog(action = ..., resourceType = ...) was declared and placed on all state-altering controller and service methods (e.g., document uploads, version deletions, permission changes, video room joins).
2. **AOP Aspect Interceptor (AuditLoggingAspect):**
   Using @Around advice, the aspect intercepts method execution. It extracts the authenticated user's details from the SecurityContextHolder, captures the remote IP address from the HttpServletRequest, records execution duration, and writes an immutable record to the audit_logs database table.
3. **Forensic Immutability:**
   The audit_logs database entity is marked with @Immutable. Database-level table grants prevent UPDATE or DELETE operations on the audit_logs table for all standard application roles, guaranteeing that even compromised administrator accounts cannot rewrite historical access records.

3.2 Other Responsibilities During the Internship

Beyond core full-stack programming, my internship encompassed a wide range of professional engineering responsibilities:

1. Agile Scrum Ceremonies and Sprint Planning

I actively participated in all team agile ceremonies:

- **Daily Standups:** Delivered concise daily progress reports, articulated technical blockers, and coordinated dependencies with backend and frontend teammates.
- **Sprint Planning Sessions:** Contributed to breaking down high-level requirements from the *KMS Requirements Specification* into actionable user stories, estimating story points, and defining acceptance criteria.
- **Sprint Retrospectives:** Shared observations regarding local development environment bottlenecks and collaborated on continuous improvement of our testing workflows.

2. Peer Code Reviews and Collaborative Pair Programming

In accordance with INSA's development standards, no code was merged into the main branch without peer review:

- I reviewed pull requests authored by colleagues, verifying adherence to TypeScript typing standards, clean architectural separation in Spring Boot, and proper exception handling.
- Engaged in collaborative pair programming sessions with fellow developers, particularly when untangling subtle React state re-rendering issues in the Next.js document library and configuring Spring Security filter chains.

3. Technical Documentation and System Specification

I authored and maintained comprehensive technical documentation throughout the project:

- Documented all RESTful endpoints using OpenAPI 3.0 / Swagger UI annotations (@Operation, @ApiResponse), providing live API documentation at http://localhost:8081/swagger-ui.html.
- Drafted the *Deployment and Operations Guide*, detailing container orchestration steps, Keycloak realm import commands, and database migration procedures using Flyway.

4. Security Vulnerability Assessments and Code Audits

Working within a cybersecurity agency provided the opportunity to participate in security auditing activities:

- Utilized static application security testing (SAST) tools (SonarQube) to scan backend Java and frontend TypeScript codebases for common vulnerabilities, including OWASP Top 10 risks (SQL injection, XSS, insecure deserialization).
- Verified dependency trees using mvn dependency:check and npm audit to detect and remediate known Common Vulnerabilities and Exposures (CVEs) in third-party libraries.

3.3 Technical Knowledge and Skills from Coursework Beneficial for Assignment

The rigorous academic curriculum provided by the **Department of Computer Science and Engineering** at **Adama Science and Technology University (ASTU)** was foundational to my performance during this internship. Theoretical principles learned in lecture halls directly informed practical engineering decisions:

Table 3.6: Mapping of ASTU Academic Courses to Industrial Project Tasks

ASTU Academic Course	Core Theoretical Concepts Learned	Direct Industrial Application in INSA-KMS
Data Structures and Algorithms (CSE 2101 / ECE 2102)	Inverted indexing, hash functions, tree traversals, computational complexity ($O(1)$, $O(\log n)$).	Designed PostgreSQL GIN full-text index structures; evaluated SHA-256 hash collision resistance; optimized multi-level document folder traversal algorithms.
Database Systems (CSE 3201)	Relational algebra, 3NF normalization, ACID transaction properties, foreign keys, B-trees.	Designed the 18-table normalized PostgreSQL database schema; managed referential integrity cascades; configured transactional isolation levels (@Transactional(readOnly = true)).
Object-Oriented Programming & Design Patterns (CSE 2102)	Encapsulation, Polymorphism, Inversion of Control (IoC), Dependency Injection, Singleton, Factory, Repository, and DTO patterns.	Developed modular Spring Boot microservices; implemented repository and service abstractions; decoupled API payloads from entities using Data Transfer Objects (DTOs); utilized Spring AOP.
Computer Networks and Data Communication (CSE 3301)	TCP/IP protocol stack, HTTP/HTTPS methods, TLS handshakes, WebSocket, XMPP, network sockets.	Configured secure CORS/CSRF headers; engineered RESTful HTTP endpoints; analyzed XMPP BOSH signaling packets and WebRTC media streams for Jitsi Meet conferencing.

CHAPTER 4: REFLECTION AND CONCLUSION

4.1 How Did This Internship Fit Your Career Goals?

From the outset of my undergraduate studies in the Department of Computer Science and Engineering at **Adama Science and Technology University (ASTU)**, my overarching professional ambition has been to become a **Senior Software Security Architect and Enterprise Systems Engineer**. My long-term career vision is to design, engineer, and defend sovereign, mission-critical digital infrastructures that protect national digital assets, ensure data sovereignty, and empower technological self-reliance. Securing a software engineering internship at the **Information Network Security Administration (INSA)** aligned seamlessly with these aspirations across several dimensions:

1. **Immersion in Sovereign, Mission-Critical Computing:**
   Unlike commercial software environments where development speed often takes precedence over system resilience and security rigor, INSA operates under the premise that software failures can compromise national security. Working on the **INSA Knowledge Management System (INSA-KMS)** placed me directly at the intersection of enterprise software engineering and defensive cybersecurity.
2. **Mastery of Enterprise-Grade Architecture:**
   The project required architecting a production-scale, multi-tier distributed system using industry-standard enterprise frameworks: **Spring Boot 3.3**, **Next.js 14**, **Keycloak 26.7.2**, **PostgreSQL 15**, and **Jitsi Meet / Prosody XMPP**. Engaging with these complex technologies allowed me to transition from academic textbook theories to building high-throughput, low-latency, resilient software.
3. **Deep Practical Grounding in Cryptographic Protocols and Identity Federation:**
   Implementing federated Single Sign-On (SSO) using OpenID Connect (OIDC) and OAuth 2.0, verifying RS256 asymmetric cryptographic signatures against Keycloak JWKS endpoints, and generating mathematically sound RFC 7519 HMAC-SHA256 room admission tokens for WebRTC video conferences bridged the gap between abstract cryptographic theory and practical software implementation.
4. **Professional Engineering Standards:**
   The internship exposed me to enterprise-grade version control workflows, automated CI/CD pipelines, container orchestration with Docker, and rigorous peer code reviews. This experience equipped me with the professional discipline, code quality standards, and collaborative mindset expected of a software engineer in high-performance engineering organizations.

4.2 Did Your Career Goals Change as a Result of This Internship Experience?

While my core aspiration to pursue a career in software engineering and cybersecurity remained steadfast, the two months of intensive technical immersion at INSA significantly **refined, specialized, and matured** my career trajectory:

1. **Shift from Generic Full-Stack Development to Secure Systems Architecture:**
   Prior to the internship, my focus was largely centered on full-stack application development—building user interfaces and CRUD database operations. However, witnessing firsthand how subtle architectural flaws, improper token validation, or unauthenticated signaling endpoints can compromise an entire organization shifted my passion toward **Secure Systems Engineering and DevSecOps**. I realized that true engineering excellence lies in building architectures that are *secure by design* and *resilient by default*.

2. **Recognition of the Importance of Sovereign Infrastructure:**
   The experience deepened my awareness of the strategic necessity of sovereign technology. Relying on foreign, proprietary commercial cloud platforms for classified national communications introduces unacceptable risks of data sovereignty violation, external surveillance, and vendor lock-in. This realization inspired me to focus on building self-hosted, sovereign, open-source enterprise solutions tailored to the security needs of African public institutions and critical infrastructures.
3. **Commitment to Advanced Research and Graduate Studies:**
   Identifying real-world research problems during the internship—such as the need for localized multilingual semantic search in low-resource languages like Amharic and zero-trust dynamic access control—ignited an interest in applied research. As a result, I now intend to pursue Master of Science (M.Sc.) studies in **Cybersecurity and Distributed Systems** following the completion of my B.Sc. degree at ASTU.

4.3 Discuss Your Feelings About the Value of This Internship

Reflecting upon the two months spent at INSA, I regard this internship as the most transformative, intellectually stimulating, and practically valuable phase of my university education. The experience delivered profound value across multiple areas:

1. Demystifying Enterprise Complexity

In an academic setting, students typically construct isolated applications where security is minimal, databases contain dozens of sample records, and deployments run on localhost. At INSA, I engaged with production-grade complexity: multi-container network bridges, asynchronous document extraction pipelines, database indexing over dense text vectors, and strict identity federation boundaries. Successfully integrating these heterogeneous components demystified enterprise software engineering and replaced academic imposter syndrome with technical confidence.

2. Appreciation for Software Craftsmanship and Defensive Engineering

The high engineering standards enforced by my industrial supervisor and the INSA team permanently transformed my coding philosophy. I no longer view writing code merely as satisfying functional requirements; I view it as crafting robust, defensive, maintainable, and audit-compliant software. Concepts such as defensive parameter sanitization, fail-closed access boundaries, transactional consistency, and comprehensive automated test coverage have become second nature to my engineering workflow.

3. Sense of Purpose and National Contribution

Contributing directly to a platform that safeguards classified intelligence and empowers cybersecurity specialists across Ethiopia provided an immense sense of purpose. Knowing that the lines of code I wrote in Spring Boot, the Keycloak security policies I established, and the video conferencing modules I secured directly protect institutional memory within the nation's premier cyber defense agency instilled deep professional pride and civic responsibility.

4.4 What Challenges Have You Faced During the Internship?

Undertaking the development of a complex, secure enterprise system within a demanding two-month timeline naturally involved substantial technical, architectural, and environmental challenges:

Challenge 1: Keycloak 26.7.2 Token Synchronization and Spring Security Filter Chains

- **The Problem:** Keycloak 26 introduced modern architectural updates and deprecated several legacy adapter configurations in favor of pure Spring Security 6 OAuth2 Resource Server implementations. Initially, when the Next.js frontend exchanged authorization codes for JWTs and passed them to the Spring Boot backend, the backend rejected requests with 401 Unauthorized errors due to a mismatch in role extraction paths and JWKS public certificate caching failures during rapid local restarts.
- **Resolution Strategy:** I conducted a deep dive into the official Spring Security 6 reference manual and RFC 7519 standards. I engineered a custom KeycloakJwtGrantedAuthoritiesConverter to parse nested roles from realm_access.roles in the JWT claims payload. Additionally, I configured robust retry mechanisms and health checks in the backend to ensure that Spring Boot initializes its JwtDecoder only after Keycloak's JWKS endpoint is fully responsive.

Challenge 2: Jitsi Meet / Prosody XMPP BOSH Cryptographic Handshake and Room Scoping

- **The Problem:** Integrating a sovereign video conferencing engine required interfacing with a containerized Jitsi Meet stack. While open-access video calls functioned effortlessly, enforcing strict security boundaries—where only invited participants possessing a cryptographic JWT could enter a specific room—proved challenging. The Prosody XMPP server's mod_auth_token module initially rejected backend-generated JWTs with ambiguous not-allowed errors. Furthermore, preventing the meeting admission JWT from leaking into browser URLs as a query parameter (?jwt=...) was critical to prevent security exposure in access logs and proxy caches.
- **Resolution Strategy:** I systematically analyzed Prosody's Lua authentication source code to discover the exact claim structure expected: verifying that the secret key was correctly shared via environment variables, the issuer (iss) and audience (aud) matched configuration values, and the room claim matched the lowercase session identifier. To eliminate URL token leakage, I re-architected the frontend integration to pass the admission token through secure session memory directly to the Jitsi external API constructor, completely eliminating query parameter leakage while ensuring clean URLs.

Challenge 3: Asynchronous Apache Tika Ingestion and Database Performance Under Heavy Load

- **The Problem:** When users uploaded large multi-megabyte PDF or Word documents containing extensive technical diagrams and text, parsing the file synchronously within the HTTP upload controller caused thread starvation, slow response times, and HTTP 504 gateway timeouts. Furthermore, performing full-text searches across large document tables resulted in sequential table scans that slowed query response times as data volume increased.
- **Resolution Strategy:** I decoupled the document ingestion pipeline into an asynchronous event-driven architecture using Spring's @Async thread pools. File upload requests return an immediate 201 Created response once the physical file and initial metadata are persisted. In the background, worker threads parse the text using Apache Tika, update the search_vector column, and commit the transaction. To eliminate sequential database scans, I implemented PostgreSQL Generalized Inverted Indexes (GIN) on the search_vector column, reducing search latency from hundreds of milliseconds to under 15 milliseconds.

Challenge 4: Managing Complex Agile Deadlines and Scope

- **The Problem:** Balancing full-stack feature development across six interconnected subsystems within 45 working days, while authoring comprehensive automated tests and university documentation, posed substantial time management pressures.

- **Resolution Strategy:** I applied strict Agile Scrum discipline, utilizing daily issue tracking boards, maintaining modular branch management in Git, prioritizing core security and data management features before building supplementary UI embellishments, and seeking immediate supervisory feedback whenever technical blockers arose.

4.5 Discuss Your Strengths and Areas for Improvement as Self-Evaluation

Engaging in honest, critical self-evaluation is essential for continuous professional development. Based on my performance, peer interactions, and supervisory feedback during the internship, I identify the following core strengths and areas for future improvement:

Identified Strengths:

1. **Strong Analytical and Algorithmic Problem Solving:**
   Demonstrated ability to break down complex architectural requirements—such as multi-user debounced search selectors, cryptographic token generation, and database full-text indexing—into modular, computationally efficient implementations.
2. **Rapid Technical Adaptability and Self-Directed Learning:**
   Quickly mastered unfamiliar technologies within the first two weeks of the internship, including Keycloak 26 identity administration, Spring Security 6 OAuth2 resource server filters, Next.js 14 App Router paradigms, and XMPP BOSH signaling protocols.
3. **Commitment to Software Security and Code Quality:**
   Consistently applied defensive engineering practices, writing clean, well-commented, and robust code. Authored 20 automated unit and integration tests, verified fail-closed security invariants, and ensured zero SQL injection, XSS, or credential leakage vulnerabilities existed in delivered components.
4. **Professional Work Ethic, Punctuality, and Collaboration:**
   Maintained 100% attendance and punctuality throughout the 45 working days, actively participated in daily agile standups, welcomed constructive code review feedback, and collaborated harmoniously with engineers across backend, frontend, and infrastructure domains.

Areas for Future Improvement:

1. **Advanced Frontend Animation and Micro-Interaction Design:**
   While my frontend implementations are functional, accessible, and clean, my proficiency in crafting sophisticated micro-interactions, fluid CSS animations, and complex custom SVG visualizations could be enhanced to elevate the overall user experience.
2. **Large-Scale Distributed Systems and Asynchronous Message Brokers:**
   In the current INSA-KMS implementation, asynchronous tasks are managed using Spring's in-memory thread pools. For massive enterprise scale spanning tens of thousands of concurrent users across nationwide offices, integrating distributed message streaming platforms like Apache Kafka or RabbitMQ would provide superior horizontal scalability and fault tolerance.
3. **Advanced Container Orchestration with Kubernetes:**
   During the internship, container orchestration was achieved using Docker and Docker Compose. Developing deep operational expertise in Kubernetes (K8s), including Helm charts, service meshes (Istio), and automated horizontal pod autoscaling, represents a key technical milestone for my post-graduate professional development.

4.6 Conclusion

The two-month industrial internship at the **Information Network Security Administration (INSA)** has been an invaluable, defining milestone in my academic and professional development as a software engineer.

Through the engineering of the **INSA Knowledge Management System (INSA-KMS)**, I successfully transformed theoretical principles acquired in ASTU classrooms into an enterprise-grade, secure, and production-verified software solution. The delivered system successfully resolves INSA's historical challenges of institutional knowledge fragmentation, provides sub-second semantic document retrieval, enforces strict role-based and classification-based access control, enables sovereign video collaboration, and provides tamper-evident audit logging.

Beyond the concrete technical artifacts produced, the internship instilled within me a deep appreciation for software security craftsmanship, agile teamwork, and the vital role that sovereign technology plays in defending national cyberspace. I conclude this internship tenure equipped with enhanced technical competence, professional resilience, and an enduring commitment to advancing Ethiopia's technological and cybersecurity capabilities.

CHAPTER 5: RECOMMENDATIONS

5.1 Personal Opinion of the Company and the Internship

The **Information Network Security Administration (INSA)** is an exemplary, high-caliber national institution that stands at the technological vanguard of Ethiopia's digital transformation and cyber sovereignty. My two-month tenure within the Software Research & Development and Cybersecurity Infrastructure Directorate provided a rare and privileged vantage point into how a mission-critical federal agency operates.

My personal opinion of INSA and the internship program encompasses several key observations:

1. **A Culture of National Purpose and Technical Excellence:**
   The organizational culture at INSA is defined by a deep sense of patriotic duty, rigorous discipline, and continuous innovation. Engineers, researchers, and administrators approach their daily duties with the clear awareness that their work directly shields Ethiopia's vital national infrastructures—from telecommunications and banking to energy and government services—against sophisticated domestic and transnational cyber threats. This high-stakes environment fosters an inspiring, purpose-driven atmosphere that elevates the focus and output of every team member.
2. **Access to Advanced, Cutting-Edge Infrastructure:**
   Unlike many public sector institutions where technology stacks remain legacy-bound, INSA actively invests in modern, enterprise-grade tools, secure computing hardware, containerized virtualization environments, and advanced software engineering frameworks. Working alongside senior engineers with deep expertise in cryptography, distributed systems, and network security provided immense intellectual enrichment.
3. **Exceptional Mentorship and Open Collaboration:**
   Despite the heavy operational responsibilities borne by senior staff, the willingness of my industrial supervisor and team members to mentor, review code, explain complex cryptographic and architectural principles, and provide continuous feedback was extraordinary. Interns are not

relegated to menial administrative tasks; rather, they are entrusted with substantive, challenging software projects that directly serve institutional needs.

In summary, the internship at INSA was an enriching, highly professional experience that set a gold standard for what industrial training should offer to aspiring engineering students.

5.2 Recommendations for Improving the Internship Program

While the internship experience was profoundly successful, there are always opportunities for institutional refinement. Based on my observations during the two-month period, I respectfully submit the following constructive recommendations to both **Adama Science and Technology University (ASTU)** and the **Information Network Security Administration (INSA)**:

5.2.1 Recommendations for Adama Science and Technology University (ASTU)

1. **Incorporate Enterprise DevOps, Microservices, and Containerization into the Core Curriculum:**
   While ASTU provides an exceptional foundation in data structures, algorithms, object-oriented programming, and relational database theory, modern industrial engineering demands immediate proficiency in containerization (**Docker**, **Kubernetes**), continuous integration and continuous deployment (**CI/CD** pipelines), and API gateway orchestration. Introducing dedicated laboratory modules on Dockerized microservices and automated deployment pipelines in third-year coursework will significantly shorten the onboarding learning curve for interns entering enterprise environments.
2. **Expand Practical Laboratory Coursework on Enterprise Security Protocols and Identity Federation:**
   Most academic software projects implement basic username-and-password authentication stored directly in local database tables. In real-world enterprise architectures, this pattern is strongly discouraged in favor of federated identity protocols such as **OpenID Connect (OIDC)**, **OAuth 2.0**, and **SAML 2.0**. Offering elective or core laboratory modules focused on configuring identity brokers (such as Keycloak), understanding JSON Web Token (JWT) cryptographic signatures, and applying Spring Security filter chains will prepare students for high-security enterprise environments.
3. **Establish Structured Pre-Internship Industry Alignment Workshops:**
   The School of Electrical Engineering and Computing should organize intensive one-week technical preparation bootcamps prior to the commencement of the internship period. These workshops could cover Git collaboration workflows, enterprise project management tools (Jira, Trello), code review etiquette, and industry-standard documentation practices.

5.2.2 Recommendations for the Hosting Organization (INSA)

1. **Streamline Technical Toolchain Onboarding and Sandboxed Environment Setup:**
   Due to INSA's stringent security controls and network segmentation policies, provisioning local development dependencies, obtaining container images, and configuring local proxy settings required several days of administrative coordination at the start of the internship. Establishing standardized, pre-configured offline development virtual machine (VM) images or containerized dev-containers containing approved SDKs (Java, Node.js, Maven, Docker) would enable future interns to begin productive coding within their first 48 hours.

2. **Organize Mid-Term Cross-Directorate Technical Showcases:**
   Given INSA's multidisciplinary structure—spanning cyber operations, cryptographic research, forensics, and software R&D—instituting a formal mid-term technical presentation session where interns from different directorates present their projects would foster cross-pollination of ideas, enhance public speaking skills, and provide a broader perspective on the agency's overall cyber defense mission.

3. **Formalize a Transition Pipeline for High-Performing Interns:**
   INSA should establish a structured post-internship recruitment pipeline. Exceptional graduating seniors who successfully deliver major enterprise systems (such as INSA-KMS) could be offered provisional graduate employment or final-year B.Sc. capstone sponsorship, allowing the agency to retain top-tier homegrown engineering talent directly from leading science and technology universities like ASTU.

5.3 Should We Consider the Company for Future Student Placement?

I unequivocally and wholeheartedly recommend that the School of Electrical Engineering and Computing and the Department of Computer Science and Engineering at Adama Science and Technology University continue and expand student placements at the Information Network Security Administration (INSA).

The justification for this strong recommendation rests on three compelling pillars:

1. Match with ASTU's Mandate as an Elite Science and Technology Institution

ASTU is designated as a Centre of Excellence in Science and Technology, tasked with producing engineers capable of leading Ethiopia's technological advancement. Placing students at INSA—the nation's highest-tier cybersecurity and technical intelligence agency—ensures that ASTU candidates are tested against the most demanding, advanced, and security-critical software engineering problems in the country. It bridges academic excellence with national strategic imperatives

.

2. High-Impact, Production-Grade Project Allocations

Unlike placements where students are limited to passive observation or basic maintenance, INSA systematically entrusts interns with real, consequential software engineering challenges. As demonstrated by the **INSA-KMS** project, interns are empowered to architect full-stack distributed systems, configure enterprise identity brokers, implement complex database indexing, and write production code that directly solves organizational bottlenecks.

3. Culture of Mentorship and Professional Discipline

INSA provides an environment where professional integrity, security awareness, and engineering rigor are modeled by seasoned practitioners. Students placed at INSA return to ASTU not only with vastly enhanced coding capabilities, but also with sharpened professional ethics, punctuality, teamwork skills, and an enduring sense of responsibility toward national digital defense.

Expanding the institutional partnership between ASTU and INSA—through formalized internship quotas, joint research initiatives, and sponsored capstone projects—will yield immense mutual dividends for both institutions and the nation at large.

REFERENCES / BIBLIOGRAPHY

1. **Adama Science and Technology University (ASTU).** (2024). *Internship Guideline for Interns Only*. School of Electrical Engineering and Computing, Department of Computer Science and Engineering, Adama, Ethiopia.
2. **Information Network Security Administration (INSA).** (2013). *Re-establishment of the Information Network Security Administration Proclamation No. 808/2013*. Federal Negarit Gazette of the Federal Democratic Republic of Ethiopia, 20th Year No. 6, Addis Ababa, Ethiopia. Available at: https://www.insa.gov.et.
3. **Federal Democratic Republic of Ethiopia.** (2006). *Council of Ministers Regulation to Provide for the Establishment of the Information Network Security Administration Regulation No. 130/2006*. Addis Ababa, Ethiopia.
4. **Keycloak Project Team.** (2024). *Keycloak Server Administration and OpenID Connect Configuration Guide (Version 26.7)*. Red Hat, Inc. Available at: https://www.keycloak.org/documentation.
5. **PostgreSQL Global Development Group.** (2024). *PostgreSQL 15 Documentation: Chapter 12. Full Text Search and GIN Indexes*. Available at: https://www.postgresql.org/docs/15/textsearch.html.
6. **Apache Software Foundation.** (2024). *Apache Tika: A content analysis toolkit (Version 2.9)*. Available at: https://tika.apache.org/.
7. **Jitsi Community.** (2024). *Jitsi Meet and Prosody Authentication Architecture with JWT Tokens*. Available at: https://jitsi.github.io/handbook/docs/devops-guide/secure-domain.
8. **Vercel, Inc.** (2024). *Next.js 14 App Router and Server Components Documentation*. Available at: https://nextjs.org/docs.